

MAGNNET: Multi-Agent Graph Neural Network-based Efficient Task Allocation for Autonomous Vehicles with Deep Reinforcement Learning

Lavanya Ratnabala, Aleksey Fedoseev, Robinroy Peter, and Dzmitry Tsetserukou

Abstract—This paper addresses the challenge of decentralized task allocation within heterogeneous multi-agent systems operating under communication constraints. We introduce a novel framework that integrates Graph Neural Networks (GNNs) with a centralized training and decentralized execution (CTDE) paradigm, further enhanced by a tailored Proximal Policy Optimization (PPO) algorithm for multi-agent deep reinforcement learning (MARL). Our approach enables unmanned aerial vehicles (UAVs) and unmanned ground vehicles (UGVs) to dynamically allocate tasks efficiently without necessitating central coordination in a 3D grid environment. The framework minimizes total travel time while simultaneously avoiding conflicts in task assignments. For the cost calculation and routing, we employ reservation-based A^* and R^* path planners. Experimental results revealed that our method achieves a high 92.5% conflict-free success rate, with only a 7.49% performance gap compared to the centralized Hungarian method, while outperforming the heuristic decentralized baseline based on a greedy approach. Additionally, the framework exhibits scalability with up to 20 agents with allocation processing of 2.8 s and robustness in responding to dynamically generated tasks, underscoring its potential for real-world applications in complex multi-agent scenarios.

Keywords — Multi-agent system, Task Allocation, Multi-agent Deep Reinforcement Learning, Graph Neural Network, CTDE, Scalability

I. INTRODUCTION

The deployment of multi-agent systems (MAS) comprising autonomous robots and drones has witnessed remarkable growth across diverse applications, including disaster response, agricultural monitoring, surveillance, and warehouse logistics. The inherent advantages of enhanced scalability, distributed operations, and parallel execution render MAS highly appealing for addressing complex real-world challenges and traversing multi-terrain environments [1]. However, significant bottlenecks persist in effectively assigning tasks to the most suitable agents, particularly in scenarios where tasks emerge dynamically and agents experience constrained communication capabilities.

Traditional task allocation methods can be broadly categorized into centralized and decentralized approaches. Centralized methods, for example, the Hungarian algorithm [2], offer the advantage of producing globally optimal task assignments; however, they require continuous communication of each agent's state to a designated central node. This

The authors are with the Intelligent Space Robotics Laboratory, CDE, Skolkovo Institute of Science and Technology, Bolshoy Boulevard 30, bld. 1, 121205, Moscow, Russia. {lavanya.ratnabala, aleksey.fedoseev, robinroy.peter, d.tsetserukou}@skoltech.ru

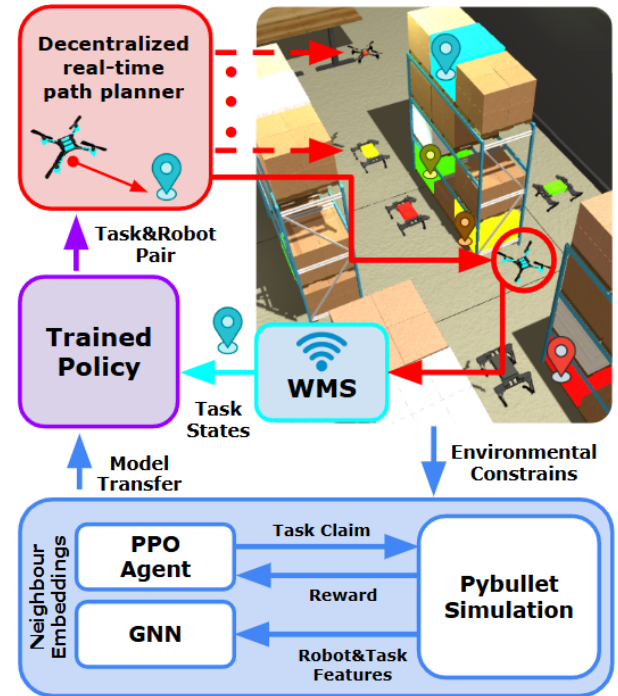


Fig. 1: Multi-vehicle task allocation scenario with a heterogeneous swarm.

dependency introduces several critical limitations, e.g., communication overhead, restricted scalability, and susceptibility to single points of failure. In contrast, decentralized methods exhibit greater robustness against agent failures and communication disruptions while also offering improved scalability as the number of agents increases [3], but often yield suboptimal solutions. This can result in conflicts or idle tasks, especially under conditions of high partial observability.

To address these challenges, we introduce MAGNNET, an innovative framework that synergizes multi-agent deep reinforcement learning (MARL) with Graph Neural Networks (GNNs) within a CTDE paradigm. During the training phase, a global critic observes the comprehensive state of the system, subsequently updating the agents' policies using the PPO algorithm to facilitate global optimization. In the execution phase, agents operate independently, relying exclusively on local observations augmented by GNN-enhanced message passing. The GNN plays a pivotal role in shaping relational embeddings for each agent by analyzing local neighborhood information and agent-to-task interactions, which empowers agents to make informed decisions regarding the acceptance or rejection of incoming tasks. The proposed framework

can extend beyond UAVs and UGVs, adapting to facilitate autonomous vehicle taxi matching, thereby reducing passenger waiting times while preventing simultaneous pickups by multiple vehicles.

II. RELATED WORKS

Multi-agent task allocation (MATA) approaches can be broadly categorized into optimization-based methods, market-based mechanisms, and learning-based approaches. Furthermore, distinctions between centralized and decentralized task allocation.

A. Optimization-Based Approaches

Optimization techniques, e.g., the linear sum assignment problem (LSAP) and the Hungarian algorithm, have been widely applied in multi-agent task allocation [4]. While these methods provide optimal solutions, they often rely on centralized computation, rendering them unsuitable for large-scale real-time applications. Ismail et al. [5] proposed a novel decentralized-based Hungarian method to solve this problem. These methods extend the Hungarian approach to decentralized settings by ensuring task allocation remains optimal as long as agent networks remain connected. Kong et al. [6] proposed a new optimization strategy that combines the improved particle swarm optimization and the greedy IPSO-G algorithm. Moreover, two different stochastic approaches, the Genetic Algorithm (GA) and the Ant-Colony Optimization (ACO) algorithm, were introduced by [7] to solve the multi-robot task allocation problem. Peter et al. [8] introduced a swarm intelligence-based task allocation method utilizing decentralized decision-making and subgoal-based path formation.

B. Market-Based Mechanisms for Task Allocation

Market-based methods, particularly auction-based approaches have been widely used for decentralized task allocation. For example, Zhong et al. [9] proposed an auction-based driver-passenger matching system that optimizes assignment stability and fairness by having drivers bid based on profitability. Liu et al. [10] extended these methods to multi-UAV systems by incorporating Dubins path-based flight cost estimation, while the Harmony Drone Task Allocation (DTA) method [11] integrates a consensus-based auction with a gossip strategy to balance task urgency and resource availability while minimizing communication load. Although these approaches offer high adaptability, their efficiency depends heavily on accurate bid valuation models.

C. Multi-Agent Deep Reinforcement Learning (DRL) in Task Allocation

The integration of DRL into MATA has led to promising advancements, particularly through MARL frameworks. Agrawal et al. [12] introduced an attention-inspired DRL method for warehouse-based task allocation that optimizes decision-making efficiency and scalability. Recent studies explore Multi-Agent Proximal Policy Optimization (MAPPO) to enhance collaborative decision-making and mitigate non-stationary challenges in multi-agent environments [13].

Additionally, it incorporates message pooling and weight scheduling mechanisms to enhance agent communication, improving efficiency in cooperative decision-making. Lowe et al. [14] use Q-learning to do cooperative and competitive tasks. QMIX is a CTDE approach that shows [15] promising results in cooperative tasks.

D. Graph Neural Networks (GNNs) for Decentralized Task Allocation

Recent advances in graph-based learning methods have demonstrated potential for improving decentralized task allocation. GNNs enable robots to model and process inter-agent relationships, allowing task allocations to be computed based on local graph structures rather than centralized control. Goarin et al. [4] proposed DGNN-GA, a decentralized GNN-based goal assignment method that optimizes communication efficiency in multi-robot planning. Heterogeneous multi-agent systems pose additional complexities due to differences in robot capabilities, sensor modalities, and mobility constraints. The GATAR framework [16] introduced a graph-based task allocation framework for multi-robot target localization, specifically designed for heterogeneous robot systems. Furthermore, Blumenkamp et al. [17] develop a real-world framework for deploying decentralized GNN-based policies in multi-robot systems, facilitating seamless sim-to-real transfers.

Despite advancements in MATA, several challenges remain unaddressed or partially solved. Many DRL-based solutions struggle to generalize to larger teams due to the inherent non-stationarity of multi-agent environments. While GNN-based models enhance decentralized coordination.

III. MULTI-VEHICLE TASK ALLOCATION APPROACH

A. Problem Formulation

We consider N agents, denoted by $\{a_1, a_2, \dots, a_N\}$, and M tasks, $\{T_1, T_2, \dots, T_M\}$. Each task T_j is characterized by a location in $\mathbb{R}^3$, denoted by l_j , and a status such as *Waiting* or *Assigned*. Each agent a_i has a position $p_i \in \mathbb{R}^3$ and a status. Statuses are categorized as *idle*, which means that the agent does not act; *accept*, which means it is willing to accept the task; and *assign*, which is the final decision to assign the task. Once they complete the task, the state changes to *complete*.

The c_{ij} is the cost for a_i to complete T_j , i.e., travel time to complete the task. $x_{ij} \in \{0, 1\}$ is the combinatorial variable, where $x_{ij} = 1$ implies that a_i is assigned to T_j . In our approach, we don't directly solve x_{ij} ; rather, each agent selects an action (request or reject) for any given task.

If multiple agents request the same T_j within one decision step, the environment assigns T_j to whichever agent has the smallest cost c_{ij} . The overall objective is to minimize $\sum_{i=1}^N \sum_{j=1}^M c_{ij} x_{ij}$, subject to each task being assigned to at most one agent. The environment is modeled as a heterogeneous graph $\mathcal{G}$ consisting of two node types, i.e., agent node and task node. Each agent node stores position $p_i \in \mathbb{R}^3$, agent status, and cost estimates $\{c_{ij}\}$ for all

available tasks T_j . The task node stores position $\mathbf{l}_j \in \mathbb{R}^3$ and task status.

Agent-Agent Edges: An edge is added between two agent nodes a_i and a_j if the Euclidean distance $\|\mathbf{p}_i - \mathbf{p}_j\|$ is below a defined communication threshold. This mechanism allows agents within local proximity to share information through GNN message passing. Such connections help the model encode spatial coordination and improve robustness in decentralized decision-making by enriching agent embeddings with neighborhood context.

Agent-Task Edges: Each edge (a_i, T_j) exists between agent a_i and task T_j to allow the agent to evaluate its suitability for completing the task. In our case, every agent node is connected to every task node, ensuring that each agent has access to global task information. This fully connected agent-task subgraph is crucial for enabling informed decisions at the agent level.

Meanwhile, by restricting full connectivity to the agent-task pairs and maintaining sparse agent-agent connections, our approach strikes a balance between global insight and efficient local communication. The multi-layer convolutional Graph Neural Network (GNN) is applied to transform node and edge features into final *embeddings*. After L message-passing iterations, the embedding for the agent node a_i is given by:

$$\mathbf{z}_i^{(L)} = \text{GNN}\left(\mathbf{z}_i^{(0)}, \{\mathbf{z}_k^{(0)} : k \in \mathcal{N}(i)\}, E\right), \quad (1)$$

where $\mathbf{z}_i^{(0)}$ is the initial feature vector of the node a_i , $\mathcal{N}(i)$ is the set of its neighbors, and E is the edge attributes. Each agent then incorporates $\mathbf{z}_i^{(L)}$ alongside its other local observations when selecting actions.

B. Reinforcement Learning Formulation for Multi-Agent Task Allocation

We formulate the decentralized task allocation problem as a partially observable multi-agent Markov decision process (POMDP) under the centralized training and decentralized execution (CTDE) paradigm. The objective is to minimize the total travel cost incurred by a team of N agents servicing M tasks.

a) Agent Observation: At each discrete time step t , an agent $a_i \in \{1, \dots, N\}$ receives a local observation vector:

$$o_i^t = \left[s_i^t, \{c_{ij}^t\}_{j=1}^M, \mathbf{z}_i^{(L)} \right], \quad (2)$$

where $s_i^t \in [-1, 1]$ is the normalized agent status (e.g., idle, accepted, assigned), and c_{ij}^t is the normalized cost to go from agent a_i to task T_j . Although we do not explicitly append the agent's 3D position $\mathbf{p}_i^t$ in the final observation vector, it is used internally by the environment to compute each c_{ij}^t . Lastly, $\mathbf{z}_i^{(L)} \in \mathbb{R}^d$ is the GNN embedding that captures local neighborhood relationship.

b) Action Space: Each agent chooses a discrete action:

$$a_i^t \in \{0, 1, \dots, M\}, \quad (3)$$

where $a_i^t = 0$ corresponds to rejecting all task requests in that timestep, and $a_i^t = j$ (for $j \in \{1, \dots, M\}$) corresponds to requesting assignment to task T_j .

c) Policy and Advantage Estimation: Each agent follows a decentralized policy $\pi_i(a_i^t | o_i^t; \theta_i)$, parameterized by weights θ_i . To stabilize training, we employ a centralized value function $V_{\text{central}}^\pi(s^t)$, where s^t is the global state comprising all agents' positions, task states, and assignments. The policy gradient is guided by the estimated advantage:

$$A_i^t = r_i^t + \gamma V_{\text{central}}^\pi(s^{t+1}) - V_{\text{central}}^\pi(s^t), \quad (4)$$

where, r_i^t is the immediate reward of the agent i at time t , and $\gamma \in [0, 1]$ is the discount factor that balances immediate versus future returns.

d) Reward Function: The reward signal received by each agent is defined as:

$$r_i^t = \begin{cases} +R_{\text{assign}}, & \text{if } a_i \text{ wins the requested task,} \\ -R_{\text{conflict}}, & \text{if } a_i \text{ competes but loses,} \\ -R_{\text{idle}}, & \text{if } a_i \text{ stays idle,} \\ +R_{\text{global}}, & \text{if all tasks are assigned successfully.} \end{cases} \quad (5)$$

After training, the centralized critic V_{central}^π is discarded. During deployment, each agent independently executes its learned policy π_i , relying solely on local observations and GNN-derived context, enabling scalable and decentralized decision-making in dynamic environments.

C. Model Architecture and Training Strategy

1) Neural Network Architecture: The GNN consists of two graph convolutional layers, each with 6 hidden units. These layers enable agents to aggregate information from neighboring agents within a communication range and from task nodes, facilitating more informed task allocation decisions. Each layer is followed by a ReLU activation function to introduce non-linearity and improve representational capacity. The input features to the GNN include agent positions, agent statuses, cost estimates for available tasks, and task positions. After message passing, the final agent embeddings $\mathbf{z}_i^{(L)} \in \mathbb{R}^6$ are obtained, encoding both local information and relational dependencies.

Each agent's policy network is a multi-layer perceptron (MLP) that takes as input the concatenation of $\mathbf{z}_i^{(L)}$ with local observations, producing a probability distribution over the discrete action space. The input layer has a dimension of $M + 7$, where $M + 1$ corresponds to local observations consisting of the agent's status and cost estimates for M tasks, while the remaining six dimensions correspond to the GNN-generated embedding. The network contains a hidden layer with 128 neurons, followed by a ReLU activation, and an output layer that applies a softmax activation to produce a probability distribution over available actions.

The central value network follows a similar MLP architecture but processes a global representation of the environment. It takes as input the concatenation of all agent observations, capturing the full system dynamics. The network includes a hidden layer with 128 neurons followed by a ReLU activation and outputs a single scalar value estimate, representing the overall value of the system state. The policy and value networks share feature representations and are optimized using

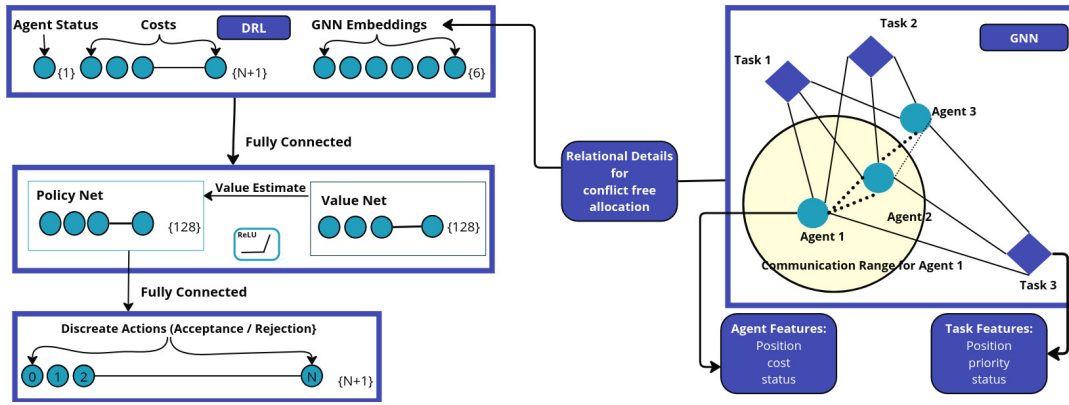


Fig. 2: MAGNET architecture for decentralized task allocation. Each agent uses local observations (status, costs) and GNN-based embeddings from agent-agent (dotted) and agent-task (solid) edges to select actions via its policy network.

the PPO algorithm. The overall neural network architecture is shown in Fig. 2.

2) *Learning Algorithm*: We developed an algorithm based on the PPO approach of Ray RLlib, a scalable reinforcement learning framework designed for distributed multi-agent training. The policy and value networks are optimized using the Adam optimizer with a learning rate of 1×10^{-5} . Training is conducted with a batch size of 512, using a stochastic gradient descent (SGD) mini-batch size of 64 and 10 SGD iterations per update. To encourage exploration and prevent premature convergence, an entropy coefficient of 0.05 is applied. The discount factor is set to $\gamma = 0.99$ to ensure that agents optimize long-term task allocation efficiency. Generalized advantage estimation (GAE) with $\lambda = 0.95$ is employed to reduce variance while maintaining bias in advantage computation, leading to more stable training updates.

Stable learning is typically achieved between 200k and 500k environment interactions. During training, the centralized critic leverages global state information to refine individual policies, ensuring that agents learn strategies that improve long-term task efficiency. Once training is complete, the centralized critic is discarded, and each agent operates in a fully decentralized manner. Agents rely solely on their local sensing capabilities and GNN-derived relational embeddings to make real-time autonomous decisions.

D. Simulation Setup and Path Planning

1) *Simulation Environment*: We developed a custom PyBullet-based simulator to train and evaluate decentralized task allocation in a heterogeneous multi-agent system. Fig. 3 shows our simulation environment. The environment is a three-dimensional, grid-based world where agents navigate to complete dynamically assigned tasks while avoiding obstacles. The simulation is integrated with Ray RLlib for reinforcement learning. We trained the model with randomized parameters, applying clipping and normalization to ensure robust real-time performance across diverse locations [18].

2) *Path Planning and Cost Computation*: Task allocation alone does not guarantee efficient execution; agents must

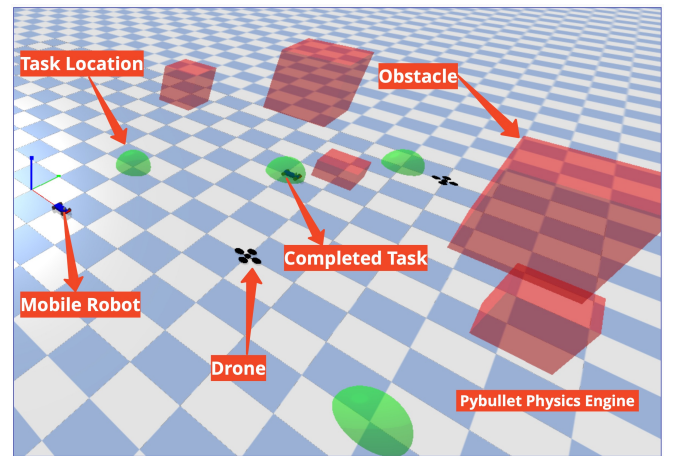


Fig. 3: Simulation environment in PyBullet. Agents navigate with dynamic task assignments and obstacles.

navigate to their assigned tasks efficiently. We utilize an A^* path planning algorithm to compute the shortest path between an agent and its assigned task, considering obstacles and other agents in the environment. The computed path is then used to calculate the task completion cost based on the agent velocity. The cost function is formulated as:

$$c_{ij} = \frac{d_{ij}}{v_i}, \quad (6)$$

where c_{ij} is the cost for the agent a_i to complete the task T_j , d_{ij} is the shortest path distance from agent a_i to task T_j , computed with A^* , and v_i is the velocity of the agent. Since multiple agents navigate toward their respective tasks, a reservation-based conflict resolution mechanism is applied to prevent collisions. Prior to moving, each agent verifies if the target position is already occupied or reserved and if an obstacle obstructs the path. In case of a conflict, an agent initiates dynamic replanning using an updated A^* search. If a conflict arises, the system resolves it through priority-based resolution: the agent with the lowest task completion cost retains its path, while others replan. If no alternative path

exists, the agent waits for the occupied space to become available. This combination of A^* for distance computation, velocity-based cost estimation, and conflict-aware navigation ensures efficient and conflict-free decentralized task execution.

IV. EXPERIMENTS

A. Experimental Setup

We use a $50 \times 50 \times 30$ m simulation environment in PyBullet, populated with N heterogeneous agents comprising both ground robots (velocity ~ 3 m/s) and aerial drones (velocity ~ 5 m/s). Up to 20 tasks appear at random locations at fixed time intervals. Agents navigate around randomly placed obstacles and compute cost-to-task estimates using an A^* path planner. All experiments were conducted in a custom PyBullet-based environment built upon the Ray RLlib MultiAgentEnv framework. Our simulation supports GNN-based local observation embedding via PyTorch Geometric. Task allocation policies were trained using PPO in a centralized training and decentralized execution setup.

We evaluated three metrics for the algorithms: total travel time, estimated as the sum of all task travel times; conflict-free success rate, defined as the fraction of tasks assigned to exactly one agent; and allocation time, measured as the time needed for agents to finalize assignments. In all experiments, the number of tasks equaled the number of agents. We compared MAGNNET against the centralized Hungarian method, which provides the optimal solution, as well as a heuristic greedy coalition auction algorithm (GCAA) [19] and Random assignment. Experiments were performed with team sizes ranging from 4 to 20 agents.

B. Results and Discussion

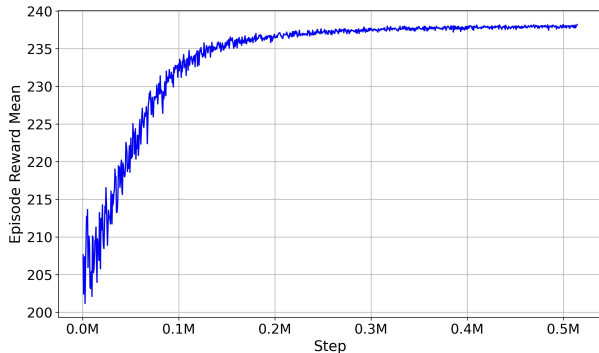


Fig. 4: Mean reward vs. training steps during learning.

1) *Training performance*: The increase in rewards shown in Fig. 4, coupled with a decrease in entropy shown in Fig. 5, indicates that agents are progressively learning to allocate tasks more efficiently over time. Initially, high entropy reflects uncertainty in decision-making due to insufficient experience, leading to suboptimal task assignments and lower rewards. However, as training progresses, agents refine their policies, resulting in consistent task selection that is reflected in the rising reward trend, which signifies that agents are

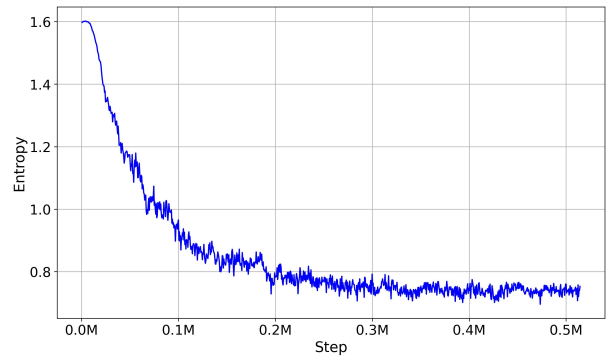


Fig. 5: Entropy vs. training steps during learning.

successfully maximizing their task allocation efficiency. The entropy reduction further confirms that agents are transitioning from an exploratory phase to a more deterministic and optimized decision-making process, leading to improved overall system performance.

2) *Cost of total travel time*: In the first scenario, we fix $M = N$ tasks that appear at the start. Table I summarizes the total travel cost achieved by each method for various numbers of agents.

TABLE I: Total Travel Cost in Static Task Scenario.

N	Hungarian	MAGNNET	Greedy	Random
4	20.3	20.3	22.5	27.9
8	60.3	61.2	65.8	72.3
12	131.9	134.7	140.5	175.7
20	254.3	321.5	383.2	423.8

Since the total travel cost is invariant to the chosen path planner, we compared the performance of MAGNNET trained on reservation-based A^* with a collision-free RRT^* approach adapted to our discrete environment. The results of the comparison are shown in Table II.

TABLE II: Mean Path Length (in m) of Two Planning Algorithms Embedded in MAGNNET.

Path Planning	N Agents			
	4	8	12	20
A^* algorithm	5.75	13.63	20.83	38.40
RRT^* algorithm	8.86	13.87	19.86	40.95

The results revealed that at a lower number of agents A^* outperformed the baseline method by 35%, however, with an increasing number of agents, the performance of both algorithms is approximately equally efficient, with variance within 6.5%. Table I shows the comparison of the total cost versus the baseline Hungarian and decentralized the Greedy approach and Random allocation. MAGNNET approach demonstrates a performance close to the centralized Hungarian baseline with an average gap of under 7.49%, whereas the Greedy and Random approaches show noticeably higher total cost, particularly as the number of agents grows.

3) *Conflict-free success rate*: We measured the percentage of tasks allocated to a singular agent. Table III shows the conflict-free success rates and approximate allocation times. The Hungarian method achieved a 100% conflict-free allocation expected from a centralized approach. Our decentralized approach maintains a high success rate of 80% to 100%, significantly outperforming the Greedy algorithm, especially for larger swarm scales.

TABLE III: Conflict-free Success Rate and Allocation Time.

Metric	Method	4	8	12	20
Success Rate (%)	Hungarian	100	100	100	100
	MAGNNET	100	100	90	80
	Greedy	90	80	80	60
Allocation Time (s)	Hungarian	0.8	1.5	2.8	5.6
	MAGNNET	0.4	0.6	1.2	2.8
	Greedy	0.3	0.3	0.5	1.2

Table III shows the conflict-free success rates against baseline Hungarian and greedy approaches. While the Hungarian method requires increased computation time with higher agent counts, our MAGNNET remains more scalable and maintains quick decision times. The Greedy method is faster but exhibits lower success rates. Results indicate that, with additional training steps and an appropriately tuned reward structure, the MAGNNET approach mitigates conflicts while retaining near-optimal allocations and handling continuous task generation. Occasional conflicts still arise in rapidly changing environments, suggesting the need for further reward shaping or improved communication strategies.

V. CONCLUSION AND FUTURE WORK

In this paper, we introduce a decentralized task allocation framework for multi-agent autonomous vehicle systems, MAGNNET, that integrates GNNs and DRL within a CTDE paradigm. Simulation results demonstrate that our approach achieves near-optimal performance only with 7.49% performance gap compared to a centralized Hungarian baseline and significantly surpasses the greedy method in terms of conflict avoidance and total travel cost, especially as the number of agents increases. In all experiments, we consider a balanced setup where the number of agents equals the number of tasks. Under this constraint, MAGNNET excels at generating one-to-one conflict-free allocations with minimal coordination overhead. Our framework achieves a 92.5% success rate in task assignments while reducing total assignment time 2.8 s for 20 tasks, compared to both the optimal (Hungarian) and greedy baselines. This result confirms MAGNNET's scalability and efficiency even in dense environments. Future directions include transferring the trained policy to physical fleets of drones and ground robots while accounting for sensor noise, real-time communication delays, and unpredictable obstacles. In the real world, this can be implemented via peer-to-peer ESP-NOW communication protocols. Further enhancements will involve refining the reward structure and incorporating attention-based GNN layers to strengthen conflict resolution in highly dynamic environments.

ACKNOWLEDGEMENTS

Research reported in this publication was financially supported by the RSF grant No. 24-41-02039.

REFERENCES

- [1] M. Zafar, R. A. Khan, A. Fedoseev, K. K. Jaiswal, and D. Tsetserukou, "Hetswarm: Cooperative navigation of heterogeneous swarm in dynamic and dense environments through impedance-based guidance," 2025. [Online]. Available: <https://arxiv.org/abs/2502.06722>
- [2] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [3] V. Singhal and D. Dahiya, "Distributed task allocation in dynamic multi-agent system," in *Proc. Int. Conf. on Computing, Communication & Automation (ICCCA)*, 2015, pp. 643–648.
- [4] M. Goarin and G. Loianno, "Graph neural network for decentralized multi-robot goal assignment," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4051–4058, 2024.
- [5] S. Ismail and L. Sun, "Decentralized hungarian-based approach for fast and scalable task allocation," in *Proc. IEEE Int. Conf. on Unmanned Aircraft Systems (ICUAS)*, 2017, pp. 23–28.
- [6] X. Kong, Y. Gao, T. Wang, J. Liu, and W. Xu, "Multi-robot task allocation strategy based on particle swarm optimization and greedy algorithm," in *Proc. IEEE Joint Int. Information Technology and Artificial Intelligence Conf. (ITAIC)*, 2019, pp. 1643–1646.
- [7] M. Shelkamy, C. M. Elias, D. M. Mahfouz, and O. M. Shehata, "Comparative analysis of various optimization techniques for solving multi-robot task allocation problem," in *Proc. Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, 2020, pp. 538–543.
- [8] R. Peter, L. Ratnabala, E. Y. Andrew Charles, and D. Tsetserukou, "Dynamic subgoal based path formation and task allocation: a neurofleets approach to scalable swarm robotics," in *Proc. IEEE Int. Conf. on Systems, Man, and Cybernetics (SMC)*, Kuching, Malaysia, 2024, pp. 878–883.
- [9] Y. Zhong, L. Gao, T. Wang, S. Gong, B. Zou, and D. Yu, "Achieving stable and optimal passenger-driver matching in ride-sharing system," in *Proc. IEEE Int. Conf. on Mobile Ad Hoc and Sensor Systems (MASS)*, 2018, pp. 125–133.
- [10] J. Liu, Z. Zhang, T. Xu, and C. Yan, "Multi-uav united task allocation via extended market mechanism based on flight path cost," in *Proc. IEEE Int. Conf. on Unmanned Systems (ICUS)*, 2024, pp. 50–55.
- [11] M. Eser and A. E. Yilmaz, "A gossip-based auction algorithm for decentralized task rescheduling in heterogeneous drone swarms," *IEEE Transactions on Aerospace and Electronic Systems*, pp. 1–25, 2025.
- [12] A. Agrawal, A. S. Bedi, and D. Manocha, "Rtaw: An attention inspired reinforcement learning method for multi-robot task allocation in warehouse environments," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2023, pp. 1393–1399.
- [13] Z. Da, "Research on multi-agent communication and collaborative decision-making based on deep reinforcement learning," 2023. [Online]. Available: <https://arxiv.org/abs/2305.17141>
- [14] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Int. Conf. on Neural Information Processing Systems (NIPS)*, 2017, pp. 6382–6393.
- [15] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 21, no. 1, January 2020.
- [16] J. Peng, H. Viswanath, and A. Bera, "Graph-based decentralized task allocation for multi-robot target localization," *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 10 676–10 683, 2024.
- [17] J. Blumenkamp, S. Morad, J. Gielis, Q. Li, and A. Prorok, "A framework for real-world multi-robot systems running decentralized gnn-based policies," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2022, pp. 8772–8778.
- [18] R. Peter, L. Ratnabala, D. Aschu, A. Fedoseev, and D. Tsetserukou, "Tornadodrone: Bio-inspired drl-based drone landing on 6d platform with wind force disturbances," in *Proc. IEEE Int. Conf. on Robotics and Biomimetics (ROBIO)*, 2024, pp. 516–521.
- [19] M. Braquet and E. Bakolas, "Greedy decentralized auction-based task allocation for multi-agent systems," in *Modeling, Estimation and Control Conf. (MECC)*, 2021, pp. 675–680.